VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Santhibastawad Road,
MachheBelagavi - 590018, Karnataka, India

DBMS LABORATORY WITH MINI PROJECT (21CSL55) REPORT

ON

"BLOOD DONOR MANAGEMENT SYSTEM"

Submitted in the partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION SCIENCE AND ENGINEERING

For the Academic Year 2023-2024

Submitted by

ROHAN MR	1JS21IS085
RAJATH P PISE	1JS21IS081
PRATHAM BABU G	1JS21IS074
SHRAVAN KUMAR	1JS22IS410

Under the Guidance of

Mr. Chetan R, Assistant Professor, Dept. of ISE, JSSATEB

Mrs. Manasa D S, Assistant Professor, Dept. of ISE, JSSATEB

2023-2024

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

JSS ACADEMY OF TECHNICAL EDUCATION

JSS Campus, Dr.Vishnuvardhan Road, Bengaluru-560060

JSS ACADEMY OF TECHNICAL EDUCATION

JSS Campus, Dr.Vishnuvardhan Road, Bengaluru-660060

DEPARTMENT OF INFORMATION SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the Mini project work entitled "**BLOOD DONAR MANAGEMENT SYSTEM**" carried out by Mr./Ms. **ROHAN M R, PRATHAM BABU G, RAJATH PISE AND SHRAVAN KUMAR** bearing **USN 1JS21IS085, 1JS21IS074, 1JS21IS081, 1JS22IS410** is a student of JSS ACADEMY OF TECHNICAL EDUCATION BENGALURU in partial fulfilment for 5th Semester DBMS Laboratory with mini project in INFORMATION SCIENCE AND ENGINEERING of the Visvesvaraya Technological University, Belagavi during the academic year 2023-24. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The mini project report has been approved as it satisfies the academic requirements in respect of mini project work prescribed for the said degree.

Mr. CHETAN R.	**Dr. REKHA P. M.**
Assistant Professor	**Professor & Head**
Dept. of ISE, JSSATEB	**Dept. of ISE, JSSATEB**

1. **Internal Examiner** _____

2. **External Examiner** _____

TABLE OF CONTENTS

ACKNOWLEDGEMENT 5

ABSTRACT 6

CHAPTER 1
INTRODUCTION

AIM AND PROBLEM STATEMENT 7

INTRODUCTION 7

OBJECTIVES 8-9

MOTIVATION 9

CHAPTER 2

LITERATURE SURVEY

INTRODUCTION 10-12

NORMALIZATION 12-14

CHAPTER 3

REQUIREMENT SPECIFICATIONS

SOFTWARE SPECIFICATION 15
FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS 15-16

CHAPTER 4

SYSTEM DESIGN

INTRODUCTION TO SYSTEM DESIGN 17

DESIGN PLAN 17

ARCHITECTURE DIAGRAM 18

ENTITIES AND ATTRIBUTES 19

SCHEMA DIAGRAM 20

ER DIAGRAM 21

CHAPTER 5

PROJECT IMPLIMENTATION

INTRODUCTION 22

CREATING TABLES 22-27

TRIGGERS 28-29

PSEUDO CODE 29-31

CHAPTER 6

SYSTEM TESTING

INTRODUCTION 32

UNITTESTING 32-38

CHAPTER 7

RESULTS AND DISCUSSIONS

SNAPSHOTS 39-42

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENTS

CONCLUSION 43

FUTURE ENHANCEMENT 43-44

CHAPTER 9

REFERENCES

BOOK REFERENCES 45

WEB REFERENCES 45

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of the people who made it possible. So with gratitude, we acknowledge all those whose guidance and encouragement crowned my effort with success.

First and foremost we would like to thank his **Holiness Jagadguru Sri Shivarathri Deshikendra Mahaswamiji** and **Dr. Bhimasen Soragaon**, Principal, JSSATE, Bangalore for providing an opportunity to carry out the DBMS MINI PROJECT (21CS63) as a part of our curriculum in the partial fulfillment of the degree course.

We express our sincere gratitude for our beloved Head of the department, **Dr. Rekha P M**, for her co-operation and encouragement at all the moments of our approach.

It is our pleasant duty to place on record our deepest sense of gratitude to our respected guide and mini project co-ordinator **Mr. CHETAN R, Assistant Professor** for the constant encouragement, valuable help and assistance in every possible way.

We would like to thank all **ISE Department teachers** and **non-teaching staff** for providing us with their valuable guidance and for being there at all stages of our work.

ROHAN M R-1JS21IS085

RAJATH PISE-1JS21IS081

PRATHAM BABU-1JS21IS074

SHRAVAN KUMAR-1JS22IS410

ABSTRACT

Blood donation plays a crucial role in healthcare systems worldwide, ensuring a steady supply of blood products for various medical procedures. However, the efficiency of blood donor management systems significantly impacts the availability and accessibility of blood products. This abstract introduces an innovative Blood Donor Management System (BDMS) designed to enhance the effectiveness and sustainability of the blood supply chain.

The proposed BDMS leverages advanced technologies such as cloud computing, data analytics, and mobile applications to streamline the entire process of blood donation, from donor recruitment to inventory management and distribution. Key features include an intuitive donor registration interface accessible via web and mobile platforms, automated appointment scheduling, real-time monitoring of blood inventory levels, and intelligent allocation algorithms to optimize blood distribution based on demand and urgency.

Furthermore, the BDMS incorporates robust data analytics capabilities to analyze donor demographics, behaviour patterns, and health records, facilitating targeted donor recruitment campaigns and personalized donor engagement strategies. By harnessing the power of predictive analytics, the system can anticipate future blood supply needs, enabling proactive measures to prevent shortages and ensure adequate stock levels.

The implementation of the BDMS promises several benefits, including improved efficiency in donor management operations, enhanced blood supply chain visibility, and increased donor satisfaction through a seamless donation experience. Moreover, by optimizing resource allocation and reducing wastage, the system contributes to the overall sustainability of the blood supply chain, ultimately saving lives and improving healthcare outcomes.

CHAPTER 1
INTRODUCTION

AIM:

To accurately track and monitor the presence or absence of the students in class.

PROBLEM STATEMENT:

The current blood donation management systems often face significant challenges, including inefficient donor tracking, suboptimal inventory management, and a lack of cohesive communication between stakeholders. Manual processes and out-dated systems contribute to delays, errors, and missed opportunities to save lives. There is a pressing need for a modernized solution that addresses these issues and streamlines the blood donation process.

INTRODUCTION:

The "Blood donation Database Management System" has been developed to override the problems prevailing in the practicing manual system. This software is supported to eliminate and, in some cases, reduce the hardships faced by this existing system. Moreover,this system is designed for the particular need of the company to carry out operations in a smooth and effective manner. The application is reduced as much as possible to avoid errors while entering the data. It also provides error message while entering invalid data.No formal knowledge is needed for the user to use this system. Thus, by this all it proves it is user-friendly. Blood Bank Management System, as described above, can lead to error free, secure, reliable and fast management system. It can assist the user to concentrate on their other activities rather to concentrate on the record keeping. Thus, it will help organization in better utilization of resources. Every organization, whether big or small, has challenges to overcome and managing the information of Donor, Blood Bank, Blood, Patient, Blood Group. Every Blood Bank Management System has different Blood Bank needs; therefore, we design exclusive employee management systems that are adapted to your managerial requirements. This is designed to assist in strategic planning, and will help you ensure that your organization is equipped with the right level of information and details for your future goals. Also, for those busy executives who are always on the go, our systems come with remote access features, which will allow you to manage your workforce anytime, at all times. These systems will ultimately allow you to better manage resources

.

OBJECTIVES:

1.**Efficient Donor Registration:**

To create a user-friendly platform for donors to easily register, providing essential informationand preferences. To establish a centralized database for comprehensive donor profiles.

2.**Real-time Inventory Management:**

To implement a system that tracks blood inventory levels in real-time, preventing shortagesand minimizing wastage. To enable quick retrieval of accurate inventory data for efficient decision-making.

3.**User Authentication and Security:**

To ensure a secure environment through multi-level user authentication, protecting sensitive donor and recipient information.

4.**Appointment Scheduling:**

To facilitate the scheduling of appointments for donors, ensuring a smooth and organized donation process. To reduce waiting times and enhance the overall donor experience.

5.**Automated Notifications and Alerts:**

To implement automated alerts for donors about upcoming appointments. To notify administrators of critical inventory levels and recipients of available blood matches.

6.**Efficient Blood Donation Management:**

Traditional methods of managing blood donation campaigns and donor information are often manual and time-consuming. Our system aims to streamline these processes, making them more efficient and less prone to errors.

7.**Increased Accessibility:**

By providing an online platform for donors to register and participate in blood donation campaigns, we aim to increase accessibility to blood donation opportunities. This can help attract more donors and ensure a steady supply of blood for medical facilities.

8.**Improved Coordination:**

Coordinating blood donation campaigns, managing donor information, and disseminating announcements can be challenging tasks. Our system provides centralized management tools to facilitate better coordination among organizers, donors, and medical facilities.

9.**Enhanced Communication:**

Effective communication is essential for the success of blood donation campaigns. Our system enables organizers to communicate important information such as campaign schedules, blood requirements, and donor guidelines to potential donors in a timely and efficient manner.

10.**Data Management and Analysis:**

Collecting and analyzing data related to blood donation campaigns and donor demographics can provide valuable insights for future initiatives. Our system includes features for data management and analysis, allowing organizers to make informed decisions and improve the effectiveness of their campaigns.

MOTIVATION:

The major goal of the blood bank management system is to keep track of blood, donors, blood groups, blood banks, and stock information. It keeps track of all information concerning blood, blood cells, stocks, and blood. Because the project is all done at the administrative level, only the administrator can see it.

CHAPTER 2

LITERATURE SURVEY

INTRODUCTION:

A database management system (DBMS) refers to the technology for creating and managing databases. Basically, DBMS is a software tool to organize (create, retrieve, update and manage) data in a database. The main aim of a DBMS is to supply a way to store up and retrieve database information that is both convenient and efficient. By data, we mean known facts that can be recorded and that have embedded meaning. Normally people use software such as DBASE IV or V, Microsoft ACCESS, or EXCEL to store data in the form of database. A datum is a unit of data. Meaningful data combined to form information. Hence, information is interpreted data – data provided with semantics. MS. ACCESS is one of the most common examples of database management software. The name indicates what database is. Database is one of the important components for many applications and is used for storing a series of data in a single set. In other words, it is a group / package of information that is put in order so that it can be easily access, manage and update.

MySQL:

MySQL is a fast, easy-to-use RDBMS being used for many small and big businesses. It is developed, marketed and supported by MySQL AB, which is a Swedish company.

MySQL is becoming so popular because of many good reasons −It is released under an open-source license. So, you have nothing to pay to use it, it is a very powerful program in its own right and handles a large subset of the functionality of the most expensive and powerful database packages. MySQL uses a standard form of the well-known SQL data language. It works on many operating systems and with many languages including PHP, PERL, C, C++, JAVA etc. and works very quickly and works well even with large data sets.

NODE.JS:

Node.js is a JavaScript runtime environment that enables server-side programming, utilizing an event-driven, non-blocking I/O model for efficient handling of concurrent connections. With Node.js, developers can write both client-side and server-side code using JavaScript,

leveraging its asynchronous programming paradigm to handle multiple tasks simultaneously. Node.js benefits from a rich ecosystem of open-source packages available through npm, facilitating the extension of application capabilities. Drawing inspiration from JavaScript's event-driven nature and the non-blocking I/O model of languages like Python, Node.js offers scalability, performance, and responsiveness for building web servers, real-time web applications, APIs, andmicroservices.

CSS:

Cascading Style Sheets (CSS) is a style sheet language used for describing the presentation of a document written in a markup language such as HTML.CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript.

CSS is designed to enable the separation of presentation and content, including layout, colors,and fonts. This separation can improve content accessibility, provide more flexibility and controlin the specification of presentation characteristics, enable multiple web pages to share formattingby specifying the relevant CSS in a separate .CSS file which reduces complexity and repetition in the structural content as well as enabling the .css file to be cached to improve the page load speed between the pages that share the file and its formatting. The name cascading comes from the specified priority scheme to determine which style rule applies if more than one rule matches a particular element. This cascading priority scheme is predictable

HTML:

Hypertext Markup Language (HTML) is the standard markup language for documents designed to be displayed in a web browser. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically and originally included cues for the appearance of the document.

HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects such as interactive forms may be embedded into the rendered page. HTML provides a means to create structured documents by denoting structural semantics for text such asheadings, paragraphs, lists, links, quotes and other items. HTML elements are delineated by

tags,written using angle brackets. Tags such as <img /> and <input /> directly introduce content into the page. Other tags such as <p> surround and provide information about document text and mayinclude other tags as sub-elements. Browsers do not display the HTML tags, but use them to interpret the content of the page.

HTML can embed programs written in a scripting language such as JavaScript, which affects the behavior and content of web pages. Inclusion of CSS defines the look and layout of content. The World Wide Web Consortium (W3C), former maintainer of the HTML and current maintainer of the CSS standards, has encouraged the use of CSS over explicit presentational HTML since 1997.

JAVASCRIPT:

JavaScript abbreviated as JS, is a programming language that conforms to the ECMAScript specification. JavaScript is high-level, often just-in-time compiled, and multi-paradigm. It has curly-bracket syntax, dynamic typing, prototype-based object-orientation, and first-class functions.

Alongside HTML and CSS, JavaScript is one of the core technologies of the World Wide Web. JavaScript enables interactive web pages and is an essential part of web applications. The vast majority of websites use it for client-side page behavior, and all major web browsers have a dedicated JavaScript engine to execute it.

As a multi-paradigm language, JavaScript supports event-driven, functional, and imperativeprogramming styles. It has application programming interfaces (APIs) for working with text, dates, regular expressions, standard data structures, and the Document Object Model (DOM).JavaScript engines were originally used only in web browsers, but they are now embedded in some servers, usually via Node.js. They are also embedded in a variety of applications created with frameworks such as Electron and Cordova.

NORMALIZATION:

Database normalization is a database schema design technique, by which an existing schema is modified to minimize redundancy and dependency of data. Normalization split a large table into smaller tables and define relationships between them to increases the clarity in organizing data.

Database normalization types:

First Normal Form (1NF):

- First normal form (1NF) deals with the `shape' of the record type
- A relation is in 1NF if, and only if, it contains no repeating attributes or groups of attributes.
- Example: The student table with the repeating group is not in 1NF

Second Normal Form (2NF):

- A relation is in 2NF if, and only if, it is in 1NF and every non-key attribute is fully functionallydependent on the whole key.

Third Normal Form (3NF):

- A relation is in 3NF if, and only if, it is in 2NF and there are no transitive functionaldependencies.
- Transitive functional dependencies arise:
- when one non-key attribute is functionally dependent on another non-key attribute
- FD: non-key attribute -> non-key attribute
- and when there is redundancy in the database

Boyce-Codd Normal Form (BCNF):

- When a relation has more than one candidate key, anomalies may result even though therelation is in 3NF.
- 3NF does not deal satisfactorily with the case of a relation with overlapping candidate keys
- i.e. composite candidate keys with at least one attribute in common.
- BCNF is based on the concept of a determinant.

Fourth Normal Form (4NF):

It is a normal form used in database normalization Introduced by Ronald Fagin in 1977, 4NF is the next level of normalization after Boyce–Codd normal form (BCNF). Whereas the second, third, and Boyce–Codd normal forms are concerned with functional dependencies, 4NF is concerned with a more general type of dependency known as a multivalued dependency

Fifth Normal Form (5NF):

It is also known as project-join normal form (PJ/NF) is a level of database normalization designed to reduce redundancy in relational databases recording multi-valued facts by isolating semantically related multiple relationships. A table is said to be in the 5NF if and only if every nontrivial join dependency in that table is implied by the candidate keys.

CHAPTER 3

REQUIREMENT SPECIFICATIONS

SOFTWARE SPECFICATION:

- Project Type: Web-Based Application
- Front-end Tech: HTML, CSS, BOOTSTRAP, JavaScript
- Database Tool: MySQL
- Back-end Tech: Node.js (with Express.js Framework),PHP
- Browser: Chrome, Firefox, Edge
- Software: Node.js, npm, MySQL Server, any text editor (eg; Visual Studio Code)

FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS:

FUNCTIONAL REQUIREMENTS:

1. User Authentication: Users should be able to authenticate themselves using a username and password. Only authenticated users should be granted access to certain functionalities, such as creating or updating records.
2. Data Management: Users should be able to perform CRUD operations (Create, Read, Update, Delete) on various entities in the system, such as admin, announce, blood, campaigndb, donor, and users.
3. Announcements Management: The system should allow administrators to create, update, and delete announcements, specifying details such as announcement text, blood group needed, date, organizer, and requirements.
4. Blood Donation Management: The system should allow users to record information about blood donors, including their personal details, blood group, contact information, and donation history.
5. Campaign Management: The system should support the management of blood donation campaigns, including details such as campaign name, organizer name, contact information, date, and description.

NON-FUNCTIONAL REQUIREMENTS:

1. Performance: The system should respond to user requests within a reasonable timeframe, ensuring that database queries and operations are executed efficiently.
2. Scalability: The system should be able to handle an increasing number of users, records, and transactions without significant degradation in performance.
3. Security: User authentication should be secure, ensuring that passwords are encrypted and stored securely in the database. Access to sensitive data should be restricted to authorized users only, with proper authentication and authorization mechanisms in place.
4. Reliability: The system should be reliable, minimizing the risk of data loss or corruption and ensuring that transactions are processed accurately and reliably.
5. Usability: The user interface should be intuitive and user-friendly, making it easy for users to navigate, input data, and perform tasks without encountering confusion or errors.
6. Data Integrity: The system should enforce data integrity constraints, ensuring that data stored in the database is accurate, consistent, and valid according to predefined rules and relationships.

CHAPTER 4

SYSTEM DESIGN

INTRODUCTION TO SYSTEM DESIGN:

System is a collection of an interrelated components that works together to achieve a purpose. System analysis is referred to the systematic examination or detailed study of a system in order to identify problems of the system, and using the information gathered in the analysis stage to recommend improvements or solution to the system. System design is an abstract representation of a system component and their relationship and which describe the aggregated functionality and performance of the system. System design is also the overall plan or blueprint for how to obtain answer to the question being asked. The design specifies which of the various type of approach. Systems design is the process or art of defining the architecture, components modules, interfaces, and data for a system to satisfy specified requirements. One could see it as the application of systems theory to product development.

DESIGN PLAN:

User Interface: The user interface includes features such as sign-up and login pages, allowing new users to register and existing users to authenticate themselves. It also provides intuitive navigation for users to access various functionalities based on their roles.

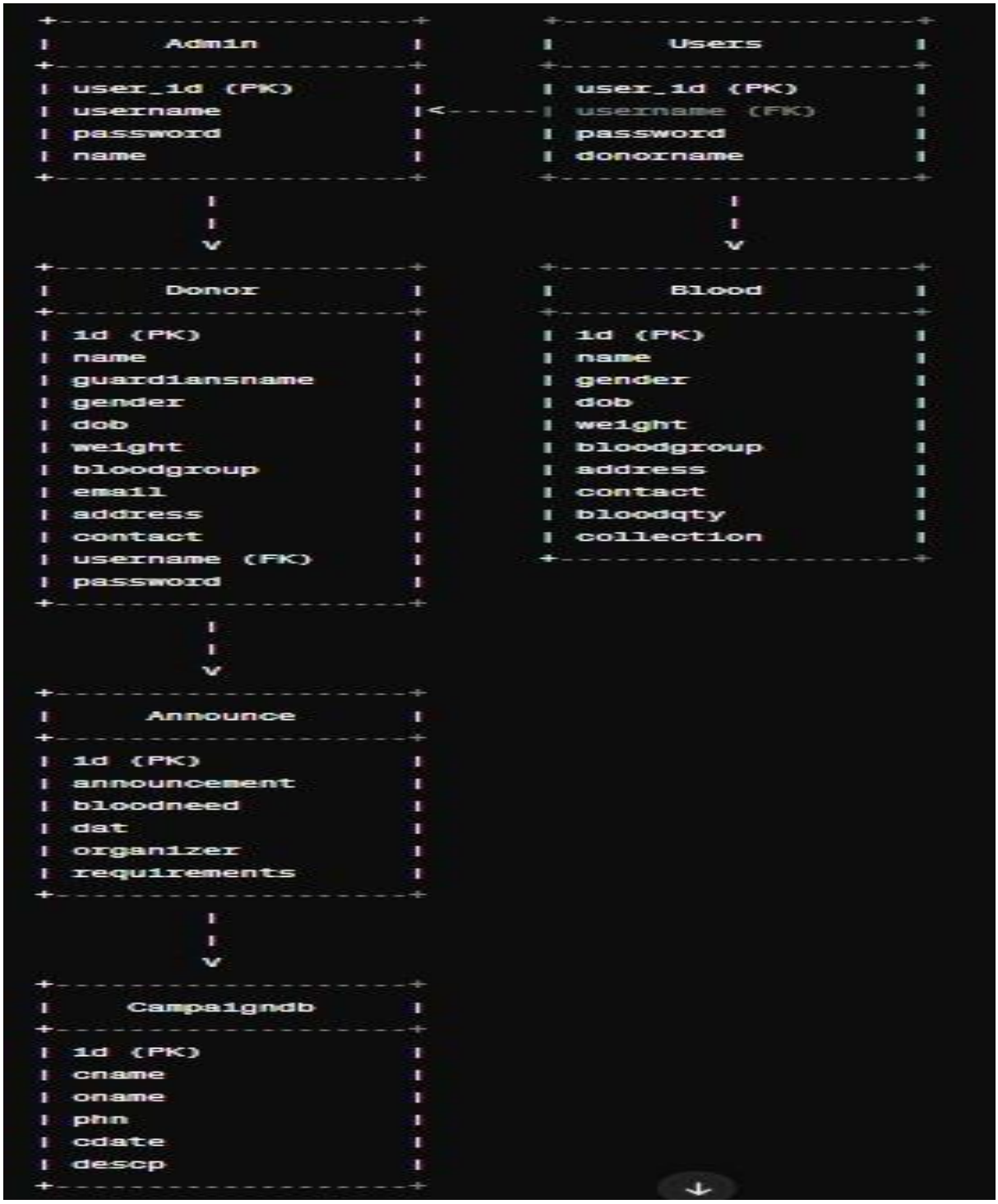
Backend Server: The backend server, powered by Node.js, facilitates communication between the frontend and the database. It handles user requests, processes data, and ensures the integrity and security of the system. In our project, the backend server also manages user authentication and authorization, directing users to specific functionalities based on their roles.

SQL Database: The MySQL database stores essential data related to users, volunteers, activities, and attendance. It provides a structured storage mechanism for efficient data retrieval, storage, and management. The database ensures data consistency and integrity, supporting various operations such as user registration, volunteer enrollment, activity management, and attendance tracking.

User Authentication/Authorization: This component manages user authentication and authorization using Role-Based Access Control (RBAC). It verifies user credentials during login and grants appropriate access rights based on predefined roles such as volunteer or administrator. RBAC ensures that users can only access functionalities relevant to their roles, enhancing system security and data privacy.

ARCHITECTURE DIAGRAM:

Architecture diagram represents the relationships between all the tables in your database. Arrows indicate the direction of the relationship, and foreign keys are denoted with (FK). For instance, the Users table has a foreign key (username) referencing the Admin table. Similarly, the Donor table has a foreign key (username) referencing the Users table.



Architecture Diagram of Blood Donor Management System

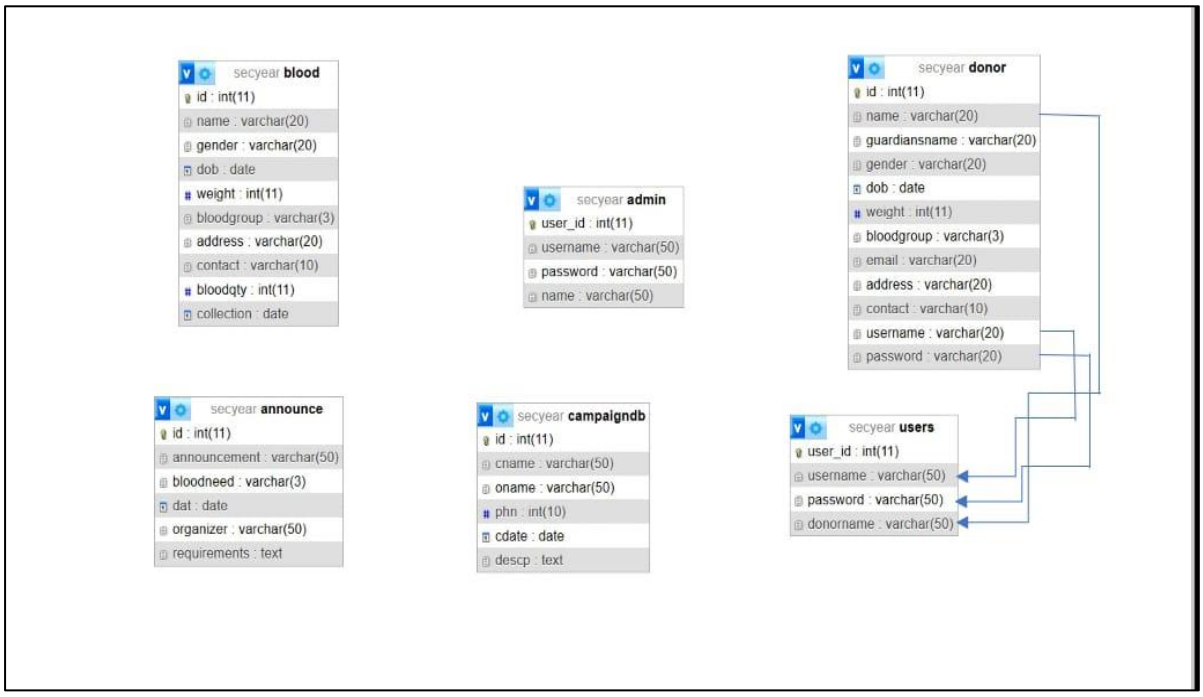
ENTITIES AND ATTRIBUTES:

Attributes define the properties of a data object and take on one of three different characteristics. They can be used to:

• Name an instance of data object.

• Describe the instance.

The Relational model for Blood Donor Management System is given in Fig



Relational Model was proposed by E.F. Codd to model data in the form of relations or tables. After designing the conceptual model of Database using ER diagram, we need to convert the conceptual model in the relational model which can be implemented using any RDBMS languages like Oracle SQL, MySQL etc. So, we will see what Relational Model is. Relational Model represents how data is stored in Relational Databases. A relational database stores data in the form of relations (tables).

SCHEMA DIAGRAM:

A database schema is the skeleton structure that represents the logical view of the entire database. A database schema defines its entities and the relationship among them. It contains a descriptive detail of the database, which can be depicted by means of schema diagrams. A schema diagram contains entities and the attributes that will define that schema. It only shows us the database design. It does not show the actual data of the database. Schema can be a single table or it can have more than one table which is related. Schema diagram of Blood Donor Management System.

Admin

User-id	Username	password	name

Announce

id	announcement	bloodneed	Date	organizer	requirements

Blood

Id	Name	dob	weight	bloodgroup	address	contact	bloodqty	collection

Campaigndb

Id	cname	oname	phn	cdate	descp

Donor

id	name	Guardians name	dob	gender	weight	email	Blood group	username	password	contact	address

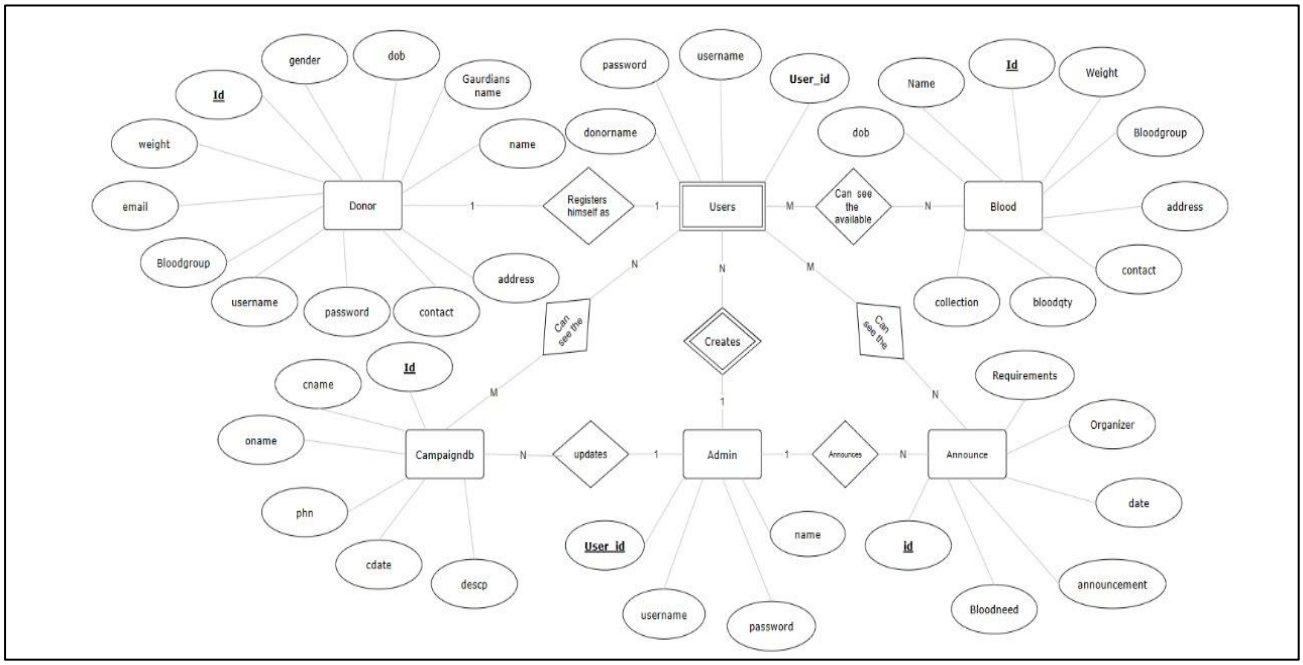
Users

User_id	donorname	username	password

The term "schema" refers to the organization of data as a blueprint of how the database is constructed. These integrity constraints ensure compatibility between parts of the schema. All constraints are expressible in the same language. A database can be considered a structure in realization of the database language. The states of a created conceptual schema are transformed into an explicit mapping, the database schema. This describes how real-world entities are modeled in the database. All the various table used are described in the following schema. The necessary Primary key's and the corresponding foreign keys are also represented.

ER DIAGRAM:

An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram). An ER model is a design or blueprint of a database that can later be implemented as a database. The main components of E-R model are: entity set and relationship set. The ER diagram shows the relationship among entity sets. An entity set is a group of similar entities and these entities can have attributes. In terms of DBMS, an entity is a table or attribute of a table in database, so by showing relationship among tables and their attributes, ER diagram shows the complete logical structure of a database. Here are the geometric shapes and their meaning in an E-R Diagram. Rectangle: Represents Entity sets. Ellipses: Attributes Diamonds: Relationship Set Lines: They link attributes to Entity Sets and Entity sets to Relationship Set Double Ellipses: Multivalued Attributes Dashed Ellipses: Derived Attributes Double Rectangles: Weak Entity Sets Double Lines: Total participation of an entity in a relationship set ER Diagram of Blood Donor Management System is shown in Fig



An entity–relationship model or the ER Diagram describes inter-related things of interest in a specific domain of knowledge. An ER model is composed of entity types and specifies relationships that can exist between instances of those entity types.

CHAPTER 5
PROJECT IMPLEMENTATION

INTRODUCTION:

Implementation is a crucial stage where the theoretical design of the Blood Donor Management System is transformed into a functional system. This phase is vital for ensuring the system meets the requirements and instills confidence in users regarding its effectiveness. Implementation can proceed only after thorough testing to ensure adherence to specifications. It involves meticulous planning, examination of the current system and its limitations, designing methods for changeover, and evaluating changeover strategies. Education and training of users, along with system testing, are essential components of implementation.

The following codes will facilitate the comprehensive implementation of our Blood Donor Management System design.

CREATING TABLES:

TABLE ADMIN

```
CREATE TABLE IF NOT EXISTS admin (
  user_id int(11) NOT NULL,
  username varchar(50) NOT NULL,
  password varchar(50) NOT NULL,
  name varchar(50) NOT NULL,
  Primary key(user_id)
)
```

Field	Type	Null	Key	Default	Extra
user_id	int(11)	NO	PRI	NULL	
username	varchar(50)	NO		NULL	
password	varchar(50)	NO		NULL	
name	varchar(50)	NO		NULL	

DESCRIPTION OF ADMIN TABLE

TABLE ANNOUNCE:

```
CREATE TABLE IF NOT EXISTS announce (
  id int(11) NOT NULL,
  announcement varchar(50) NOT NULL,
  bloodneed varchar(3) NOT NULL,
  dat date NOT NULL,
  organizer varchar(50) NOT NULL,
  requirements text NOT NULL,
  Primary key(id)
  )
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
announcement	varchar(50)	NO		NULL	
bloodneed	varchar(3)	NO		NULL	
dat	date	NO		NULL	
organizer	varchar(50)	NO		NULL	
requirements	text	NO		NULL	

DESCRIPTION OF ANNOUNCE TABLE

TABLE BLOOD:

```
CREATE TABLE IF NOT EXISTS blood (
  id int(11) NOT NULL,
  name varchar(20) NOT NULL,
  gender varchar(20) NOT NULL,
  dob date NOT NULL,
  weight int(11) NOT NULL,
  bloodgroup varchar(3) NOT NULL,
  address varchar(20) NOT NULL,
  contact varchar(10) NOT NULL,
  bloodqty int(11) NOT NULL,
  collection date NOT NULL,
  Primary key(id) )
}
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
name	varchar(20)	NO		NULL	
gender	varchar(20)	NO		NULL	
dob	date	NO		NULL	
weight	int(11)	NO		NULL	
bloodgroup	varchar(3)	NO		NULL	
address	varchar(20)	NO		NULL	
contact	varchar(10)	NO		NULL	
bloodqty	int(11)	NO		NULL	
collection	date	NO		NULL	

DESCRIPTION OF BLOOD TABLE

TABLE CAMPAIGN:

```
CREATE TABLE IF NOT EXISTS campaigndb (
 id int(11) NOT NULL,
  cname varchar(50) NOT NULL,
  oname varchar(50) NOT NULL,
  phn int(10) NOT NULL,
  cdate date NOT NULL,
  descp text NOT NULL,
  primary key(id)
)
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
cname	varchar(50)	NO		NULL	
oname	varchar(50)	NO		NULL	
phn	int(10)	NO		NULL	
cdate	date	NO		NULL	
descp	text	NO		NULL	

DESCRIPTION OF CAMPAIGNDB

```
CREATE TABLE IF NOT EXISTS donor (
  id int(11) NOT NULL,
  name varchar(20) NOT NULL,
  guardiansname varchar(20) NOT NULL,
  gender varchar(20) NOT NULL,
  dob date NOT NULL,
  weight int(11) NOT NULL,
  bloodgroup varchar(3) NOT NULL,
  email varchar(20) NOT NULL,
  address varchar(20) NOT NULL,
  contact varchar(10) NOT NULL,
  username varchar(20) NOT NULL,
  password varchar(20) NOT NULL,
  Primary key(id)
    )
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
name	varchar(20)	NO		NULL	
gender	varchar(20)	NO		NULL	
dob	date	NO		NULL	
weight	int(11)	NO		NULL	
bloodgroup	varchar(3)	NO		NULL	
address	varchar(20)	NO		NULL	
contact	varchar(10)	NO		NULL	
bloodqty	int(11)	NO		NULL	
collection	date	NO		NULL	

DESCRIPTION OF DONOR TABLE

TABLE USERS:

```
CREATE TABLE IF NOT EXISTS users (
  user_id int(11) NOT NULL,
  username varchar(50) NOT NULL,
  password varchar(50) NOT NULL,
  donorname varchar(50) NOT NULL,
  Primary key(user_id)
)
```

Field	Type	Null	Key	Default	Extra
user_id	int(11)	NO	PRI	NULL	
username	varchar(50)	NO		NULL	
password	varchar(50)	NO		NULL	
donorname	varchar(50)	NO		NULL	

DESCRIPTION OF USERS TABLE

TRIGGERS:

A trigger is a stored procedure in database which automatically invokes whenever a special event in the database occurs. For example, a trigger can be invoked when a row is inserted into a specified table or when certain table columns are being updated.

Syntax:

create trigger [trigger_name]

[before | after]

{insert | update | delete}

on [table_name]

[for each row]

[trigger_body]

TRIGGER ON BLOOD:

The Trigger update_collection_date is defined to automatically set the collection date for a new record being inserted into the blood table. So, whenever a new record is inserted into the blood table, this trigger automatically sets the collection date to the current date. This can be useful for

tracking when blood collections occur without requiring the user to manually input the date each time a new record is added

```
DELIMITER $$

CREATE TRIGGER update_collection_date

BEFORE INSERT ON blood

FOR EACH ROW BEGIN

 SET NEW. Collection=CURDATE();

 END

$$

DELIMITER ;
```

PSEUDO CODE:

Pseudocode is an informal high-level description of the operating principle of a computer program or other algorithm. It uses the structural conventions of a normal programming language, but is intended for human reading rather than machine reading.

Algorithm for Table Display :

Step1: BEGIN

Step 2: Establish connection with the database using the username and password of the database.

Step 3: Define the select query to retrieve all the values from the DBMS

Step 4: Define Default Table Model for the table and fetch the details from the database.

Step 5: END

Algorithm for Insert:

Step 1: BEGIN

Step 2: Get all the necessary values required for insertion into variable defined in the method.

Step 3: Define the query for insertion as stated above.

Step 4: Execute the Query using the (Select * from) the required table.

Step 5: END

Algorithm for Update:

Step 1: BEGIN

Step 2: Get all the necessary values required for updating the values into the variable defined in the method.

Step 3: UPDATE table name SET column1 = value1, column2 = value2, ... WHERE condition;

Step 4: Define the Query for Updating as stated above.

Step 5: Execute the Query using the executeUpdate() method defined.

Step 6: END

Algorithm for Delete :

Step 1: BEGIN

Step 2: Get the model number of the instrument which is to be deleted into a variable defined in the method.

Step 3: Delete from table_name where condition;

Step 4: Define the Query for deleting as stated above.

Step 5: Execute the Query using the executeUpdate() method defined.

Step 6: END

Algorithm for Triggers:

Step 1: create trigger [trigger_name]: Creates or replaces an existing trigger with the trigger_name.

Step 2: [before | after]: This specifies when the trigger will be executed.

Step 3: {insert | update | delete}: This specifies the DML operation.

Step 4: on [table_name]: This specifies the name of the table associated with the trigger.

Step 5: [for each row]: This specifies a row-level trigger, i.e., the trigger will be executed for each row being affected.

Step 6: [trigger_body]: This provides the operation to be performed as trigger is fired.

CHAPTER 6
SYSTEM TESTING

INTRODUCTION:

Testing plays a vital role in the success of the system. System testing makes a logical assumption that if all parts of the system are correct, the goal will be successfully achieved. Once program code has been developed, testing begins. The testing process focuses on the logical internals of the software, ensuring that all statements have been tested, and on the functional externals, that is conducted tests to uncover errors and ensure that defined input will produce actual results that agree with required results. Broadly speaking, there are at least three levels of testing: unit testing, integration testing, and system testing

UNIT TESTING:

Unit testing refers to tests that verify the functionality of a specific section of code, usually at thefunction level. In an object-oriented environment, this is usually at the class level, and the minimal unit tests include the constructors and destructors. These types of tests are usually written by developers as they work on code (white-box style), to ensure that the specific function is working as expected. One function might have multiple tests, to catch corner cases or other branches in the code. Unit testing alone cannot verify the functionality of a piece of software, but rather is used to ensure that the building blocks of the software work independentlyfrom each other.

USER AUTHENTICATION:

Test Case 1: Verify that a registered user can log in successfully with the correct username and password.

Test Case 2: Verify that an unregistered user cannot log in with invalid credentials.

Test Case 3: Verify that the system displays appropriate error messages for invalid login attempts.

Unit Testing for Login Feature: The results of unit testing on the Login Feature of the Project areshown below in Fig 6.1

SL NO.	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Adding valid user id and user-type	Valid user id entered by the user	Accept the user-type and user id	Login Successful	TEST PASSED
2	Adding user id and user-type (Not present in the user table)	Adding user id not present in the database	Detect the invalid inputs from the user	An alert message "Invalid User id and password" is displayed	TEST PASSED
3	Adding password not matching with the value present in the USERS table	User enters incorrect password for their account	"Incorrect Password" must be displayed	An alert message "Invalid User id and password" is displayed	TEST PASSED

Fig 6.1 Unit Testing for Login Feature of User Authentication

Data Management:

Test Case 4: Verify that a new record can be created for each entity (admin, announce, blood, campaigndb, donor, users) with valid data.

Test Case 5: Verify that existing records can be read from the database and displayed correctly in the user interface.

Test Case 6: Verify that existing records can be updated with new information.

Test Case 7: Verify that existing records can be deleted from the database without any errors

SL NO.	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Successful Application submission	Valid Donor details entered	Application submitted	An alert message "Application submitted" is displayed	TEST PASSED
2	Verification of Existing records	Valid user id and other credentials	Please fill the required field	An alert message "Please fill all the required fields" is displayed	TEST PASSED
3	Update Existing records with new information	Valid user id with new information to be updated	"Application submitted" must be displayed	An alert message "Application submitted" is displayed	TEST PASSED
4	Delete existing records without any error	Valid user id with valid donor name	"Application Deleted successfully"	An alert message "Application deleted successfully" is displayed	TEST PASSED

Fig 6.2 Unit Testing for Data Management:

Announcements Management:

Test Case 8: Verify that a new announcement can be created with all required fields filled out correctly.

Test Case 9: Verify that announcements are displayed correctly in the user interface with the correct details.

Test Case 10: Verify that an existing announcement can be updated with new information.

Test Case 11: Verify that an existing announcement can be deleted without any errors.

SL NO.	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	Create a new announcement with valid data for all required fields.	Enter valid data for all required fields (announcement text, blood need, date, organizer, requirements).	The new announcement should be successfully created and stored in the database.	An alert message "Announcement made" is displayed	TEST PASSED
2	View the list of announcements and ensure that they are displayed with accurate details.	Check that all announcements are listed with their respective details (announcement text, blood need, date, organizer, requirements).	All announcements should be displayed accurately in the user interface.	All announcement are displayed accurately in the table	TEST PASSED
3	Edit an existing announcement and update it with new information.	Find the announcement to be updated and select the option to edit it. Modify the existing information with new data.	The announcement should be successfully updated in the database with the new information.	An alert message "The selected announcement has been updated"	TEST PASSED
4	Delete an existing announcement from the system.	Find the announcement to be deleted and select the option to delete it. Confirm the deletion.	The announcement should be successfully deleted from the database without any errors.	The announcement is successfully deleted from database without any errors	TEST PASSED

Fig 6.3 Unit testing for announcement management.

Blood Donation Management:

Test Case 12: Verify that a new donor can be added to the system with all required fields filled out correctly.

Test Case 13: Verify that donor information is stored correctly in the database and can be retrieved when needed.

Test Case 14: Verify that donor information can be updated with new information.

Test Case 15: Verify that donor information can be deleted from the system s

SL NO.	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	**Add a new donor to the system with valid data for all required fields.**	**Enter valid data for all required fields (name, gender, date of birth, weight, blood group, contact, etc.).**	**The new donor should be successfully added to the system and stored in the database.**	**An alert message "Blood donation has been listed thank you" is displayed in user interface**	**TEST PASSED**
2	**Retrieve and verify the information of a donor from the database.**	**Search for the donor whose information was added in previous case. Retrieve and view the donor's information from the database.**	**The donor's information should be stored correctly in the database and should match the data entered during creation.**	**The donor's information is stored correctly in the database and is displayed in user interface**	**TEST PASSED**
3	**Update the information of an existing donor with new data.**	**each for the donor whose information needs to be updated. Edit the donor's information and enter new data for the fields to be updated.**	**The donor's information should be successfully updated in the database with the new data.**	**An alert message "Successfully updated" is displayed in user interface**	**TEST PASSED**

4	**Delete the information of an existing donor from the system.**	**Search for the donor whose information needs to be deleted. Select the option to delete the donor's information.**	**The donor's information should be successfully deleted from the system without encountering any errors.**	**The donor's information is successfully deleted from user interface without errors**	**TEST PASSED**

Fig 6.3 Unit testing for blood donation management.

Campaign Management:

Test Case 16: Verify that a new campaign can be created with all required fields filled out correctly.

Test Case 17: Verify that campaign details are displayed correctly in the user interface.

Test Case 18: Verify that an existing campaign can be updated with new information.

Test Case 19: Verify that an existing campaign can be deleted without any errors.

SL NO.	TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	REMARKS
1	**Create a new campaign with valid data for all required fields.**	**Enter valid data for all required fields (campaign name, organizer name, phone number, campaign date, description).**	**The new campaign should be successfully created and stored in the system.**	**An alert message "Submitted successfully" is displayed in user interface**	**TEST PASSED**
2	**View the list of campaigns and ensure that they are displayed with accurate details.**	**Check that all campaigns are listed with their respective details (campaign name, organizer name, phone number, campaign date, description).**	**All campaigns should be displayed accurately in the user interface.**	**All campaigns are displayed accurately in the user interface**	**TEST PASSED**

3	**Edit an existing campaign and update it with new information.**	**Find the campaign to be updated and select the option to edit it. Modify the existing information with new data.**	**The campaign should be successfully updated in the system with the new information.**	**An alert message "Campaign updated" is displayed in user interface**	**TEST PASSED**
4	**Delete an existing campaign from the system.**	**Find the campaign to be deleted and select the option to delete it.**	**The campaign should be successfully deleted from the system without encountering any errors.**	**The campaign details is successfully deleted from the user interface without any errors**	**TEST PASSED**

Fig 6.4 Unit testing for Campaign management.

CHAPTER 7
RESULTS

SNAPSHOTS:

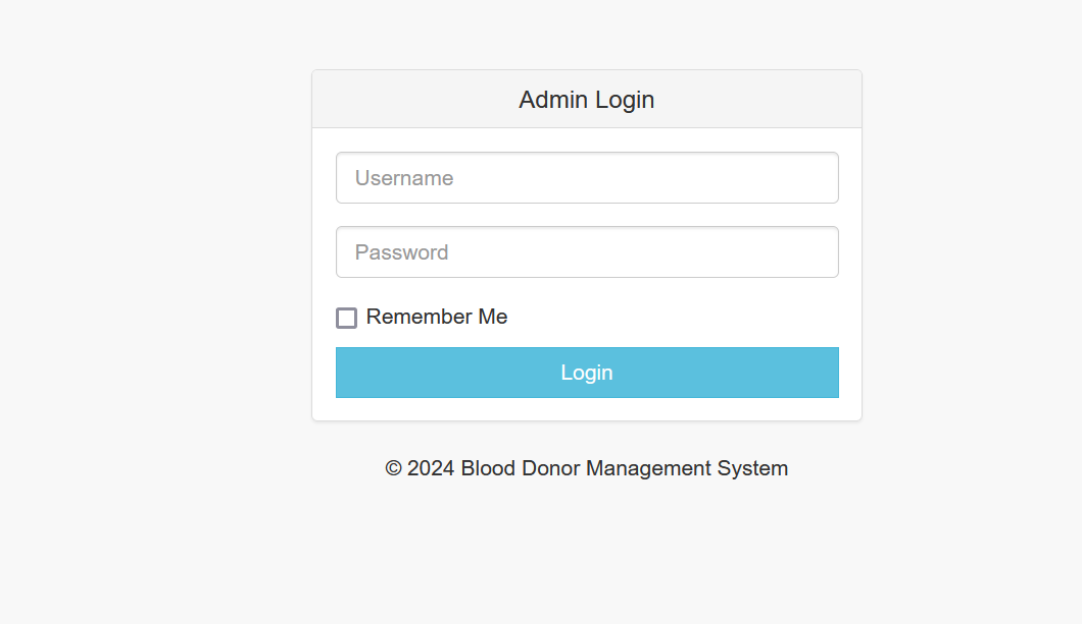


Fig.5.1 Admin login

Fig 5.2 User login

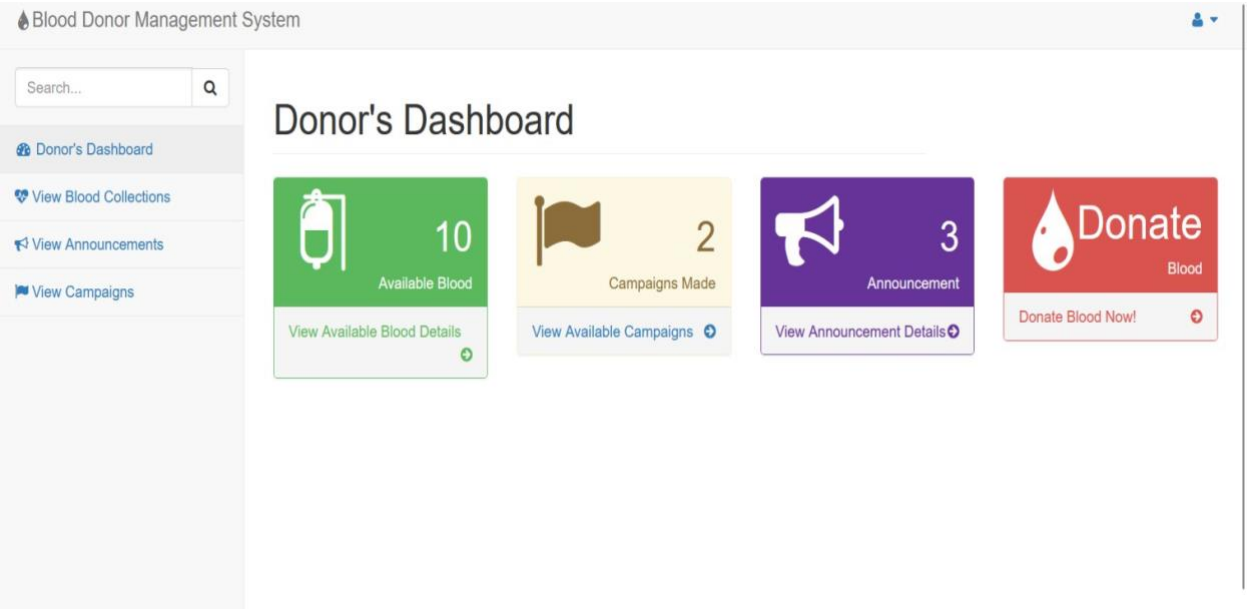


Fig 5.3 User Dashboard

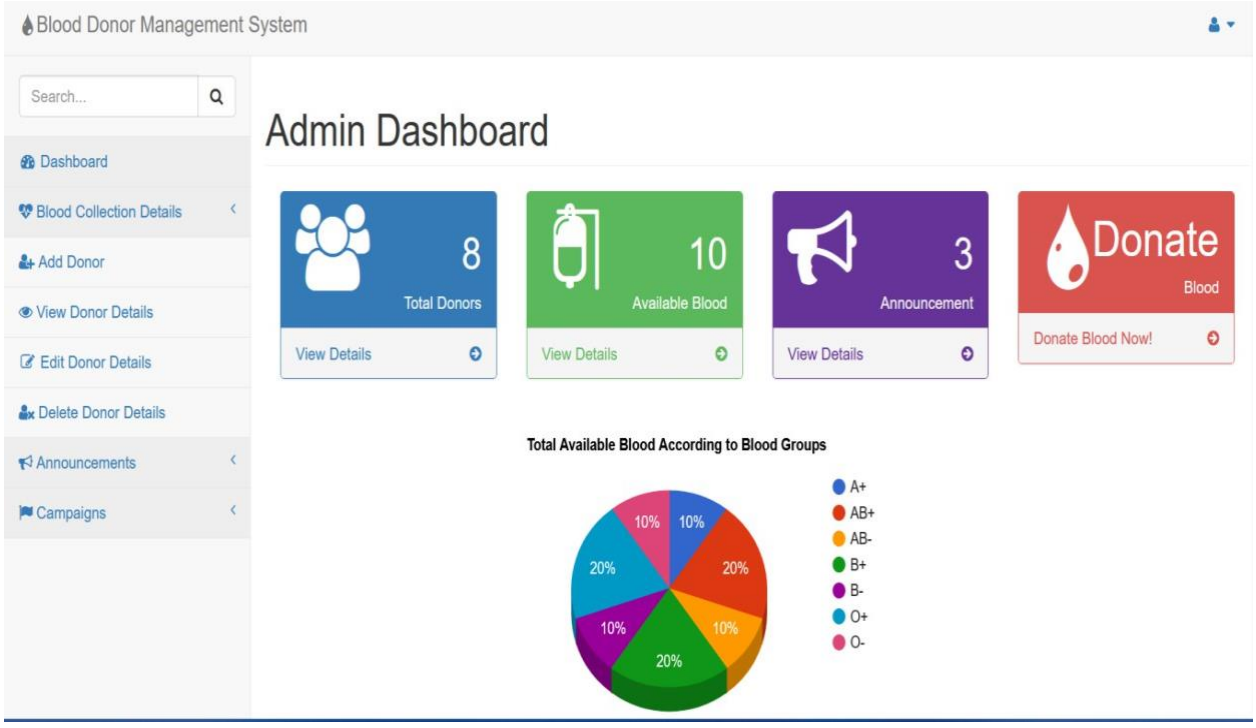


Fig 5.4 Admin Dashboard

Donor's Dashboard
View Blood Collections
View Announcements
View Campaigns

Blood Collection

Total Records of available bloods

Blood Group	Full Name	Gender	D.O.B	Weight	Address	Contact	Blood Quantity	Collection Date
B-	Shawn	M	1997-05-26	60	RT nagar	8521479633	310	2020-02-20
O+	Alia	F	1991-05-26	65	ST marks road	8555555545	360	2019-05-08
AB+	Akaay	M	1988-06-14	45	Hill road	7854447854	312	2020-12-30
B+	Sheela	F	1996-02-02	60	Whitefield	7878787850	360	2020-10-29
B+	Harry	M	1997-01-05	70	Basvangudi	8521112450	310	2023-12-09
AB-	Michael	M	1999-03-01	67	Brigade Road	8547778500	312	2023-07-12
O+	Virat	M	1990-12-24	77	Yelahanka	8545554700	360	2020-03-12
A+	Manoj	M	2002-06-02	55	Shivajinagar	7454447410	250	2023-01-05
AB+	Rose	F	1999-11-07	76	Chamrajpet	8545557854	310	2024-01-01

Fig 5.5 Blood collection

Blood Donor Management System

Search...

Donor's Dashboard
View Blood Collections
View Announcements
View Campaigns

Announcement Detail

Total Records of announcement made

Title	Blood Needed	Announcement Date	Organizer	Requirements
Cardiac Arrest	AB-	2024-01-31	RR hospital	Weight at least 50kg, No alcohol intake in 24hrs prior to donation, light meal should be taken before donation, be in good health, must be 18 years old and must have at least 3 month interval than the last donation.
URGENT CASE: Serious Accident Condition	B-	2021-03-26	City Hospital	Must be in good health and feeling very well. Must weigh at least 70 kg.
Anemia	AB+	2024-01-21	Bangalore Hospital	Make sure your health is in a good condition and Weight atleast 55kg

Fig 5.6 Announcement Detail

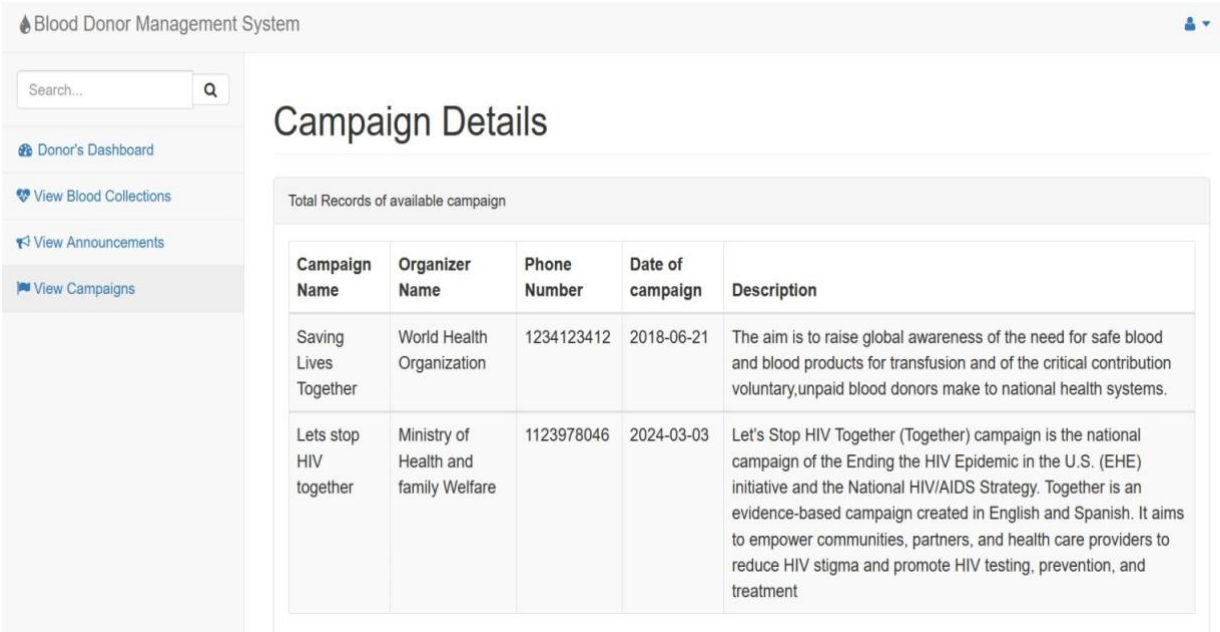


Fig 5.7 Campaign Details

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENT

CONCLUSION:

The expected outcomes include improved efficiency, enhanced user experience, secure data management, accurate reporting and analytics, integration with health records, emergency response capability, increased blood donation rates, cost-efficient operations, and most importantly, a life-saving impact. BDDMS is poised to set a new standard for effectiveness and responsiveness in blood donation management, contributing significantly to the overall improvement of the blood donation ecosystem. The proposed system provides a web-based application that is acutely useful for emergency services. It will come very useful in urgent times by providing donors information filtered by area and blood type. The system consists of a well-maintained database to keep all the registered records. It also provides news and information about the ongoing coronavirus pandemic. In the end, it provided us the knowledge regarding the latest technology required to build a web-based application. During the building of this project, it provided us an awareness of how blood donation can save lives. A database has been set up to store historical data related to donation and reception of blood and also to store data from camps so as to take future decisions based on concrete analytical results.

FUTURE ENHANCEMENT:

1. Improved Efficiency: Streamlined donor registration and appointment scheduling to reduce wait times. Real-time inventory management leading to optimal blood supply and reduced wastage.
2. Enhanced User Experience: A user-friendly interface for donors, recipients, and administrators. Automated notifications and reminders for donors, ensuring a positive and engaging experience.
3. Effective Communication: Improved communication between donors, recipients, and blood banks through the system. Timely alerts for critical inventory levels, fostering better coordination.

4. Secure Data Management: Multi-level user authentication ensuring the security of donor and recipient information .Compliance with data protection regulations for confidentiality and privacy.
5. Integration with Health Records: Seamless integration with health records for accurate compatibility checks. Enhanced medical information exchange for improved patient care.
6. Emergency Response Capability: Rapid mobilization of resources during emergencies or natural disasters. Ensured availability of blood supply in critical situations.
7. Increased Blood Donation Rates: Increased donor engagement through user-friendly features and efficient processes. A positive impact on the overall blood donation ecosystem, leading to more donations.
8. Cost-Efficient Operations: Reduction in manual processes and paperwork, leading to cost savings. Efficient resource allocation based on real-time data, optimizing operational costs.
9. Life-Saving Impact: The overall outcome aims to contribute significantly to saving lives by ensuring a well-organized, responsive, and efficient blood donation management system.
10. Accurate Reporting and Analytics: Comprehensive reports and analytics providing insights into donor demographics and blood utilization trends. Informed decision-making for blood banks and healthcare facilities based on real-time data.

CHAPTER 9

REFERENCES:

1. A Survey on Blood Bank Management System Prof. Animesh Tayal, Harshad Gahare, Akshay Patel,Sagar Jog Pratik Jain ,Jaya Dhawale Department of Computer Science & Engineering S.B. Jain Institute of Technology.
2. https://github.com/varunsardana004/Blood-Bank-And-Donation-Management-System
3. Vikas Kulshreshtha, Sharad Maheshwari. (2011).”Blood Bank Management Information System in India”, International Journal of Engineering, 1,2, 260-263.
4. https://www.researchgate.net/publication/324267381_Web_Based_Blood_Donation_Management_System_BDMS_and_Notifications.